

```

<h2>Submit With Button</h2>
<form method="post" action="/Home/SubmitWithButton">
    <input type="submit" name="submitButton" value="Save" />
    <input type="submit" name="submitButton" value="Cancel" />
</form>

@if (ViewBag.SubmitButton != null)
{
    <p>You clicked: @ViewBag.SubmitButton</p>
}

```

Testing:

- Run the application and navigate to `/Home/SubmitWithButton`.
- Click either the **Save** or **Cancel** button and check which button value is returned.

Step 7: Recap and Discussion

- **GET requests** use query strings to send data (e.g., `/Home/Details?id=5`).
- **POST requests** send form data using `<form>` tags. You can bind both primitive and complex types.
- Using the `name` and `value` attributes of a submit button helps differentiate between actions when a form has multiple buttons.

Bonus Activity (Optional)

Extend the complex type form (`SubmitComplexForm.cshtml`) to include more fields, such as address or phone number, and update the `UserModel` class accordingly.

Testing:

- Run the application and navigate to `/Home/SubmitForm`.
- Fill in the form, submit it, and see if the values are displayed.

Step 5: Handling POST Request for Complex Type

Create a form to post data using a complex type.

1. In your **Models** folder, add a new class called `UserModel.cs`.

```
public class UserModel
{
    public string Name { get; set; }
    public int Age { get; set; }
    public string Email { get; set; }
}
```

2. Add a new **View** named `SubmitComplexForm.cshtml` in **Views/Home**.

```
<h2>Submit Complex Form</h2>
<form method="post" action="/Home/SubmitComplexForm">
    Name: <input type="text" name="Name" /><br />
    Age: <input type="number" name="Age" /><br />
    Email: <input type="email" name="Email" /><br />
    <input type="submit" value="Submit" />
</form>
```

```
@if(ViewBag.User != null)
{
    <p>Name: @ViewBag.User.Name</p>
    <p>Age: @ViewBag.User.Age</p>
    <p>Email: @ViewBag.User.Email</p>
}
```

Testing:

- Run the application and navigate to `/Home/SubmitComplexForm`.
- Fill in the form and check if the values of the `UserModel` object are displayed.

Step 6: Using the `name` and `value` Attributes of a Submit Button

Create a form with multiple submit buttons.

1. Add a new **View** named `SubmitWithButton.cshtml` in **Views/Home**.

```
[HttpPost]
public IActionResult SubmitWithButton(string submitButton)
{
    ViewBag.SubmitButton = submitButton;
    return View();
}
```

Step 3: Handling GET Request for Primitive Type

Create a simple **View** that takes the `id` from the URL.

1. Right-click the **Views/Home** folder.
2. Choose **Add > View** and name it `Details.cshtml`.
3. Add the following HTML:

```
<h2>Details Page</h2>
<p>The ID you provided is: @ViewBag.Id</p>
```

Testing:

- Run the application and navigate to `/Home/Details?id=10`.
- You should see the message: "The ID you provided is: 10."

Step 4: Handling POST Request for Primitive Types

Create a form to post data using primitive types.

1. Right-click the **Views/Home** folder.
2. Add a new **View** named `SubmitForm.cshtml`.
3. Add the following HTML to display the form:

```
<h2>Submit Form</h2>
<form method="post" action="/Home/SubmitForm">
    Name: <input type="text" name="name" /><br />
    Age: <input type="number" name="age" /><br />
    <input type="submit" value="Submit" />
</form>

@if (ViewBag.Name != null)
{
    <p>Name: @ViewBag.Name</p>
    <p>Age: @ViewBag.Age</p>
}
```

In this activity, you will create an ASP.NET MVC application that retrieves data from GET and POST requests. You'll learn how to retrieve primitive and complex types and use `name` and `value` attributes in a form submission.

Step 1: Create a New ASP.NET MVC Project

1. Open **Visual Studio**.
2. Create a new **ASP.NET Core Web Application**.
3. Choose **MVC** as the template and name your project `ModelBindingExample`.

Step 2: Setting Up the Controller

Create a controller to handle requests and bind the parameters.

1. Right-click the **Controllers** folder, choose **Add > Controller**.
2. Choose **MVC Controller - Empty** and name it `HomeController`.

Add the following methods in `HomeController`:

```
using Microsoft.AspNetCore.Mvc;

public class HomeController : Controller
{
    // Step 3: GET Request for primitive type
    public IActionResult Details(int id)
    {
        // This will retrieve the 'id' from the query string or URL
        ViewBag.Id = id;
        return View();
    }

    // Step 4: POST Request for primitive type
    [HttpPost]
    public IActionResult SubmitForm(string name, int age)
    {
        // This will bind the 'name' and 'age' from the form data
        ViewBag.Name = name;
        ViewBag.Age = age;
        return View();
    }

    // Step 5: POST Request for complex type
    [HttpPost]
    public IActionResult SubmitComplexForm(UserModel user)
    {
        // The complex 'user' object is bound from form data
        ViewBag.User = user;
        return View();
    }

    // Step 6: Using Submit Button Name and Value
```